

Tutor Mx System

Gitea Sprint Handbook and Review Guide

Beginner-friendly Sprint 1-4 issue, testing, handover and deployment guide
for six Members

CURRENT STACK

React + Vite frontend | Auth0 | handwritten Node.js + Express API | Prisma | Neon PostgreSQL | Gitea main | GitHub
deployment mirror | Codecov | Cloudflare Pages | Render API

COMS3011A Project 1 | 2026

Fully updated 23 August 2026

AI Declaration: This edition was updated with ChatGPT [GPT-5.6 Sol] from the supplied handbook, the current integrated Tutor Mx source and the team's recorded workflow. The team remains responsible for checking requirements, commands and evidence before submission.

What changed in this edition

- Gitea main is the official shared code. The old handbook instructions that told Members to target a develop branch are removed.
- All six Sprint 1 Member contributions are treated as the current integrated baseline. Sprint 2-4 work builds on that baseline instead of recreating it.
- New users choose Student, Tutor or Organiser before Auth0 sign-up. Express performs server-side onboarding and creates/links the matching Neon Profile automatically.
- Existing users sign in once and are routed directly to their saved role workspace. Normal sign-in does not ask them to choose a role again.
- Organiser Tutor Management now lists self-registered Tutor accounts. Organisers search by name/email and Edit the database-backed profile; they never type an Auth0 subject/User ID.
- The exact current repository test gate (npm run check), coverage commands and local browser verification steps are documented on pages 7-9.
- The lecturer-runner rule is recorded clearly: local tests are always required; an unavailable lecturer-managed runner is an external blocker, not permission to skip tests or invent green CI.
- The deployment path is clarified: approved Gitea main -> GitHub deployment mirror -> fresh Vite build -> manual Cloudflare Production upload for the frontend.

CURRENT SPRINT 1 BASELINE

Members 1-6 are integrated in the current main-ready source. The Organiser/Tutor features use real API routes where implemented; Student overflow/volunteer remains the deliberate Sprint 1 mock boundary until Sprint 2 connects the real Student API. The Sprint 1 Save allocation control is still a planned placeholder until the allocation-write endpoint is completed.

How the team works now

Use this method in Sprint 1, 2, 3 and 4

NO DEVELOP BRANCH

Gitea main is the official shared branch. Members do normal feature work on their own branch. The Integration Lead reviews/integrates approved work into main.

Step	What to do
1. Refresh	git switch main -> git pull origin main
2. Create/use your branch	git switch -c SX/mY/short-name (or switch to your existing issue branch)
3. Work locally	Make only the changes for your issue. Keep secrets out of files/screenshots.
4. Test	Follow page 7. Then run the browser/local verification on page 8.
5. Commit your branch	Commit focused work on your branch. Do not commit directly to main.
6. Push + hand over	Push the branch to Gitea and tell the group: issue ID, branch, tests run, known limits, AI attribution.
7. Review + integrate	Reviewer checks the issue. Integration Lead integrates approved work into main and reruns the affected regression/full gate.
8. Refresh again	After main changes, everyone pulls the latest main before starting/resuming later work.

INTEGRATION LEAD EXCEPTION

When the Integration Lead performs integration/repository-maintenance work themselves, reviewed/tested changes may be committed through their integration workflow to main. This does not turn main into a normal Member working branch.

Architecture and automatic role onboarding

Layer	Beginner meaning
1. Browser	Loads Tutor Mx. New user chooses Student / Tutor / Organiser.
2. Cloudflare Pages	Serves the React + Vite frontend. Only public VITE_* configuration belongs here.
3. Auth0	Universal Login handles Google or email/password identity and email verification.
4. React onboarding	First-time account calls POST /api/auth/onboarding with the selected role + bearer token.
5. Express on Render	Verifies Auth0 identity, applies role policy and uses server-only Auth0 Management credentials when needed.
6. Prisma + Neon	Creates/reads the matching Profile and application data. Browser code never connects directly to Neon.
7. Later sign-ins	React calls /api/me and opens the saved role workspace automatically.
ONE ROLE PER TUTOR MX ACCOUNT The selected role is treated as one-time. Use separate test accounts/emails for Student, Tutor and Organiser. Do not use normal onboarding to switch an established account between roles.	

SECRETS RULE

DATABASE_URL and Auth0 Management API client secrets stay on the backend host. They must never be placed in VITE_* variables, screenshots, issues, chat, or committed files.

Very beginner guide: create an account or sign in

Action	Expected path
Create Student	Choose Student -> Create Student account -> Auth0 -> server onboarding -> /student
Create Tutor	Choose Tutor -> Create Tutor account -> Auth0 -> server onboarding -> /tutor
Create Organiser	Choose Organiser -> Create Organiser account -> Auth0 -> server onboarding -> /organiser
Existing account	Click Sign in -> Auth0 -> /api/me -> correct saved workspace. No second role choice.
Switch test role	Sign out first, then use the separate account created for the other role.
EMAIL VERIFICATION If Auth0 requires verification for an email/password account, verify the email and return to Tutor Mx. An unverified account must not be silently assigned a role.	

NORMAL SETUP IS AUTOMATIC

Do not manually create Neon Profile rows or manually assign individual Tutor Mx roles in Auth0 for ordinary new-user sign-up. Manual admin intervention is troubleshooting, not the normal Member workflow.

Organiser Tutor Management - current professional method

WHAT THE ORGANISER SEES

Search for a tutor by name or email. Select Edit to manage the tutor's details and weekly-hour limit.

- Tutor accounts appear after the Tutor signs up through the normal self-service Tutor flow.
- The Organiser does not create the Tutor identity and does not copy an Auth0 User ID/subject.
- The UI uses the tutor's internal database record automatically when the Organiser edits details.
- Marks and busy times are managed from the selected Tutor record.
- Technical explanations about Auth0/Neon are documentation concerns; they should not clutter the normal organiser interface.

SPRINT PLANNING CONSEQUENCE

Sprint 2 must not rebuild a manual tutor-create/Auth0-subject form. Sprint 2 hardens the registered-tutor management flow and completes allocation writes/approvals.

How to run the tests in VS Code / PowerShell

Every Member uses this page before handing work over

FAST RULE

From the repository root, npm run check is the current full local release-style gate. It runs Prisma generation, lint, backend type-checking, backend + frontend tests, production builds and the repository security check.

```
npm ci
npm ci --prefix frontend
npm run generate
npm run check
npm run coverage
```

Command	What it proves
npm run check	Full gate: generate -> lint -> typecheck -> tests -> build -> security check.
npm test	Backend + frontend automated tests.
npm run test:backend	Backend Vitest suite only.
npm run test:frontend	Frontend Vitest suite only.
npm run coverage	Backend + frontend coverage reports.
npm run lint	Backend + frontend lint.
npm run typecheck	Backend TypeScript check.
npm run build	Production backend + frontend builds.
npm run security:check	Tracked-file secret / repository safety check.

WHAT COUNTS AS PASSING

The command must exit successfully. Warnings are not the same as failed tests, but new warnings caused by your change should still be investigated and mentioned in the handover. Never hide a failed command.

How to check your work locally before the Integration Lead touches main

PURPOSE

This is the beginner smoke test Members run on their own branch. It answers: does my change actually work in the app before I tell the Integration Lead it is ready?

```
git switch main
git pull origin main
git switch <your-issue-branch>

# first clean setup / when dependencies changed
npm ci
npm ci --prefix frontend
npm run generate

# start frontend + backend together
npm run dev
```

- [] Open <http://localhost:5173>. The API normally runs at <http://localhost:3000>.
- [] Use the role/account relevant to your issue. For role testing, use separate Student, Tutor and Organiser test accounts.
- [] Repeat the acceptance path yourself: success case, invalid/failure case, and one relevant empty/permission case.
- [] Refresh the page and retry the action. Check that loading, error and success states remain usable.
- [] For protected flows, confirm another role cannot perform the action just by typing a URL or changing a request.
- [] Stop the dev servers with Ctrl+C. Then run `npm run check` and `npm run coverage`.
- [] Run `git status --short` and confirm only the files you intended to change are listed.

HANDOVER EVIDENCE

Send the issue ID + branch name + commands you ran + result + screenshot/API example where useful + known limitation. Do not send secrets or production personal data.

Lecturer message: tests, Actions runner and honest evidence

WHAT THE LECTURER REQUIREMENT MEANS

Automated tests are required. The course organisation/repository uses an Actions runner for CI/CD, but the lecturer-managed runner can be unavailable or queued. Local testing is still required regardless of runner state.

- Run local tests, lint/type-check/build and relevant acceptance checks before handover.
- Push your Member branch and let the reviewer/Integration Lead see the exact local evidence.
- If Gitea Actions is queued/offline because the lecturer-managed runner is unavailable, record that as an external infrastructure blocker.
- Do not claim CI/Codecov is green when it did not run. Do not weaken or remove tests just to avoid the blocked runner.
- When the runner becomes available, rerun the workflow and retain the first successful logs/evidence.
- Codecov reports coverage produced by the tests; Codecov does not run the tests itself.

SUGGESTED BLOCKER NOTE

Remote Actions queued - lecturer runner unavailable. Local npm run check / coverage evidence attached.

Member handover -> Integration Lead -> Gitea main -> GitHub mirror

```
git status
git add <only-your-files>
git diff --cached --check
git commit -m "<clear issue-focused message>"
git push -u origin <your-issue-branch>
```

- Member: do not merge your own normal feature branch into main.
- Reviewer: reproduce the issue acceptance checks and inspect the changed files.
- Integration Lead: pull latest main, integrate approved work, resolve conflicts with the author, rerun npm run check (and coverage/database checks when relevant), then push Gitea main.
- Deployment mirror: only after Gitea main is approved, sync that source to the GitHub deployment repository. GitHub is not the team source of truth.

BEFORE EVERY LATER TASK

After an integration changes main, pull main again before continuing Sprint 2/3/4 work. This prevents members from building on an old snapshot.

Manual Cloudflare Production deployment after approved integration

FRONTEND RELEASE PATH

Gitea main -> GitHub deployment mirror -> fresh local Vite build -> Cloudflare Pages Production upload.

```
git switch main
git pull origin main
cd frontend
Remove-Item -Recurse -Force dist -ErrorAction SilentlyContinue
npm run build
```

- [] Open Cloudflare Dashboard -> Workers & Pages -> tutor-mx -> Create deployment.
- [] Choose Production.
- [] Upload the newly generated frontend/dist folder.
- [] Confirm all files upload successfully, then select Save and deploy.
- [] Open tutor-mx.pages.dev and hard refresh (Ctrl + Shift + R).
- [] Run the affected role smoke test on the live site.
- [] Backend/API changes must also be deployed to the hosted Express service using the team's approved deployment mirror/host workflow.

VITE RULE

VITE_* values are embedded at build time. If production frontend environment values change, rebuild dist before uploading it.

AI policy and Global Definition of Done

AI policy

- [] Remove secrets, tokens and personal data before using an AI tool.
- [] Read, understand and test AI-generated code before committing it.
- [] If a commit contains AI-generated code, use the course-required Assisted-by: <tool>[<model>] attribution with the actual tool/model.
- [] Keep README/document AI declarations current. Never invent test evidence, stakeholder evidence or tool/model names.

Global Definition of Done - applies to every issue in Sprint 1-4

- [] The issue-specific acceptance checks pass.
- [] Page 8 local browser/behavior check is completed for the affected flow.
- [] Page 7 automated test gate is run; relevant coverage is produced.
- [] If the lecturer runner is unavailable, page 9 blocker/evidence rule is followed honestly.
- [] Only intended files are committed on the Member branch; no secrets/personal data are added.
- [] The branch is pushed to Gitea and the assigned reviewer/Integration Lead is told it is ready.
- [] The Member does not merge normal feature work to main. Integration Lead integrates approved work and reruns regression.
- [] AI attribution/declarations are correct.

Sprint 1: plan, order and handovers

3-25 August 2026 | Integrated foundation, public demo and one secure vertical path

SPRINT 1 IS THE BASELINE

Current main already contains the integrated Member 1-6 Sprint 1 foundation. Use these issue cards for traceability, regression and follow-up fixes; do not rebuild working foundations.

- Keep the automatic Auth0 -> Express onboarding -> Neon Profile method as the normal account path.
- Keep Organiser Tutor Management registered-tutor based; no Auth0 subject field.
- Keep Tutor routes own-identity based; no browser tutor-ID ownership.
- Keep Student overflow/volunteer on the Sprint 1 mock boundary until Sprint 2 real endpoints are integrated.
- Keep Save allocation as the Sprint 1 placeholder until Sprint 2 allocation-write work is ready.

Member	Sprint ownership
Member 1	Authentication, role security, notifications/search and shared release checks
Member 2	Organiser interface, allocation board, approvals, reports and scenarios
Member 3	Tutor dashboard, availability, timesheets, excuses, imports and swaps
Member 4	Student overflow plus shared responsive and accessible interface
Member 5	Handwritten Express API, validation, business rules, security and API tests
Member 6	Neon PostgreSQL, Prisma migrations, external integration, data engine, CI/deployment

EVERY MEMBER

Before handover: use page 7 for automated tests, page 8 for local browser verification, page 9 for the lecturer-runner rule, and page 12 for the Global Definition of Done.

Sprint 1 - Member 1

Authentication, role security, notifications/search and shared release checks

REQUIRED BEFORE HANDOVER

See page 7 (run tests), page 8 (check the feature locally), page 9 (Actions runner rule) and page 12 (Global Definition of Done).

SPRINT 1 BASELINE NOTE

This Member area is already part of the integrated Sprint 1 baseline. Use it to verify/repair the current behavior; do not recreate a second implementation of the same foundation.

S1-M1-1 | Self-service Auth0 sign-up, role onboarding and direct routing

Branch: s1/m1/auth-entry | Reviewer: Member 2 | 4 points | Integrates to: Gitea main

Goal: As a new or returning user, I want Tutor Mx to link my authenticated identity to exactly one role and open the correct workspace without manual setup.

Acceptance checks

- A new user chooses Student, Tutor or Organiser before Auth0 sign-up; the server creates the matching Tutor Mx role/Profile automatically.
- An existing linked user signs in once and is routed directly to /student, /tutor or /organiser without choosing a role again.
- An unverified email or onboarding conflict is handled safely; passwords and Management API credentials never enter Tutor Mx browser code.

Member tasks

- [] Maintain the role-choice entry screen and Auth0 Universal Login flow.
- [] Use POST /api/auth/onboarding for first-time linking and /api/me for later sign-ins.
- [] Keep sign-out available from all three workspaces and cover role-routing failures with tests.

Handoff: Give Member 5 the bearer-token shape and give Members 2-4 the signed-in user/role interface.

S1-M1-2 | Account settings, password reset, deletion and route protection

Branch: s1/m1/auth-lifecycle | Reviewer: Member 2 | 4 points | Integrates to: Gitea main

Depends on: S1-M1-1 and role names Organiser/Tutor/Student

Goal: As a Tutor Mx user, I want safe account recovery and protected role navigation so that I can manage access without exposing private data.

Acceptance checks

- Password-reset requests use a safe non-enumerating response and Auth0 owns the actual password workflow.
- Account deletion requires confirmation and the privileged Auth0/database deletion runs only on the backend.
- Wrong-role or unlinked access is blocked even if a user manually types a protected URL.

Member tasks

- [] Maintain Account Settings reset/deletion actions and confirmation states.
- [] Keep route guards aligned with backend role/ownership checks.
- [] Test cancel/confirm, wrong-role, missing-profile and signed-out paths.

Handoff: Give Member 5 the role matrix and give the Project Coordinator screenshots/test evidence for the security guide.

S1-M1-3 | Authentication contract and permission tests

Branch: s1/m1/auth-contract-tests | Reviewer: Member 2 | 4 points | Integrates to: Gitea main

Depends on: S1-M1-1, S1-M1-2 and S1-M5-2

Goal: As the team, we want one tested bearer-token and role contract so that every feature uses the same secure identity rules.

Acceptance checks

- Protected routes return 401 for missing/invalid tokens and 403 for valid users with the wrong role.
- A permitted token reaches the handler only when its Tutor Mx role matches the linked Neon Profile.
- Fresh first-login onboarding is handled safely even when the first token predates the newly assigned role.

Member tasks

- [] Maintain the shared bearer-token request helper.
- [] Keep backend Auth0 issuer/audience/JWKS/expiry checks covered.
- [] Add contract tests whenever a new protected route is introduced.

Handoff: Member 5 consumes the token helper/fixtures; Members 2-4 use the request wrapper rather than copying auth code.

Sprint 1 - Member 2

Organiser interface, allocation board, approvals, reports and scenarios

REQUIRED BEFORE HANDOVER

See page 7 (run tests), page 8 (check the feature locally), page 9 (Actions runner rule) and page 12 (Global Definition of Done).

SPRINT 1 BASELINE NOTE

This Member area is already part of the integrated Sprint 1 baseline. Use it to verify/repair the current behavior; do not recreate a second implementation of the same foundation.

S1-M2-1 | Organiser workspace and allocation-board foundation

Branch: s1/m2/organiser-board-shell | Reviewer: Member 1 | 4 points | Integrates to: Gitea main

Depends on: Shared navigation from S1-M4-2; mock candidate contract from

Goal: As an organiser, I want one clear workspace for courses, tutors and allocation checks so that management work is easy to follow.

Acceptance checks

- The Organiser workspace loads only for the ORGANISER role and keeps loading/empty/error states usable.
- Candidate mark, clash and hour results are visible together before selection.
- The workspace remains keyboard reachable and usable on common phone widths.

Member tasks

- [] Maintain the Organiser navigation and allocation-board shell.
- [] Keep candidate data and rule-result components aligned with the API contract.
- [] Preserve clear loading, empty, error and retry states.

Handoff: Give Member 5 a list of required fields and give Member 4 any reusable table/card needs.

S1-M2-2 | Course management and registered-tutor management UI

Branch: s1/m2/course-tutor-ui | Reviewer: Member 1 | 4 points | Integrates to: Gitea main

Depends on: S1-M2-1 and S1-M5-2 contract

Goal: As an organiser, I want to manage courses and already-registered tutors using normal account information instead of technical Auth0 identifiers.

Acceptance checks

- Signed-up Tutor accounts appear from the database automatically and can be searched by name or email.
- The Organiser never has to enter, copy or see an Auth0 subject/User ID in the normal Tutor Management UI.
- Edit uses the internal tutor database record to maintain details, weekly-hour limit, marks and busy times.

Member tasks

- [] Maintain course add/edit/remove UI with validation and confirmations.
- [] Maintain tutor search/list/edit UI for self-onboarded Tutor profiles.
- [] Keep the interface professional: no implementation-level Auth0/Neon instructions in normal organiser screens.

Handoff: Member 6 uses the final field list in migrations; Member 5 matches the request/response shapes.

S1-M2-3 | Immediate allocation-rule messages and organiser UI tests

Branch: s1/m2/allocation-rule-ui | Reviewer: Member 1 | 4 points | Integrates to: Gitea main

Depends on: S1-M2-1 and rule outputs from S1-M5-3

Goal: As an organiser, I want mark, timetable and remaining-hour checks explained before saving so that invalid choices are obvious.

Acceptance checks

- Mark threshold, clash and remaining-hours checks each show written Pass/Fail or controlled unknown wording.
- Save remains unavailable until the selected tutor passes the required checks.
- Colour is never the only way a rule result is communicated.

Member tasks

- [] Maintain the reusable rule-result component.
- [] Test pass/fail/unknown and boundary values.
- [] Keep candidate refresh and selected-tutor state stable after errors.

Handoff: Member 5 supplies stable reason codes; Member 2 maps those codes to plain language without duplicating business logic.

Sprint 1 - Member 3

Tutor dashboard, availability, timesheets, excuses, imports and swaps

REQUIRED BEFORE HANDOVER

See page 7 (run tests), page 8 (check the feature locally), page 9 (Actions runner rule) and page 12 (Global Definition of Done).

SPRINT 1 BASELINE NOTE

This Member area is already part of the integrated Sprint 1 baseline. Use it to verify/repair the current behavior; do not recreate a second implementation of the same foundation.

S1-M3-1 | Tutor dashboard using the signed-in tutor identity

Branch: s1/m3/tutor-dashboard | Reviewer: Member 4 | 4 points | Integrates to: Gitea main

Depends on: Role routing from S1-M1-2 and shared states from S1-M4-2

Goal: As a tutor, I want my own assignments and weekly-hour summary without entering a tutor ID so that another tutor cannot be selected by changing a URL.

Acceptance checks

- GET /api/tutor/dashboard derives the tutor from the verified token subject and returns only that tutor's data.
- A newly onboarded Tutor with no hour limit or allocations gets a usable no-work state rather than an error.
- Loading, empty, error and retry states remain clear.

Member tasks

- [] Maintain the protected tutor dashboard route and cards/table.
- [] Display unset weekly limits safely and never request another tutor by browser-supplied ID.
- [] Test own-data, no-work and failed-load cases.

Handoff: Give Member 5 the dashboard fields and Member 6 the query/seed-data needs.

S1-M3-2 | Tutor availability, work-log and excuse forms on real API routes

Branch: s1/m3/tutor-forms | Reviewer: Member 4 | 4 points | Integrates to: Gitea main

Depends on: S1-M3-1 and initial API contracts

Goal: As a tutor, I want to record busy times, work logs and excuses through my own authenticated routes so that my records stay accurate.

Acceptance checks

- Tutor workflow endpoints derive ownership from the token and ignore browser-supplied tutorId values.
- Invalid or impossible time/hour values are rejected with clear field messages.
- Valid actions persist through the handwritten Express API and Prisma.

Member tasks

- [] Maintain availability/time-slot form and validation.
- [] Maintain work-log and excuse submission forms.
- [] Test valid, invalid, duplicate/state and cross-user protection cases.

Handoff: Member 5 receives the payload shapes; Member 6 receives required tables and constraints.

S1-M3-3 | Tutor UI contract, responsiveness and accessibility tests

Branch: s1/m3/tutor-ui-tests | Reviewer: Member 4 | 4 points | Integrates to: Gitea main

Depends on: S1-M3-1, S1-M3-2 and S1-M4-2

Goal: As the team, we want regression tests around the tutor journey so that later changes do not break the working dashboard and forms.

Acceptance checks

- Every required tutor control remains keyboard reachable.
- Required actions are not clipped at the agreed narrow/mobile width.
- Contract or validation changes that break the current tutor journey fail tests.

Member tasks

- [] Keep dashboard/form component tests current.
- [] Cover labels, focus order and responsive layout.
- [] Run the shared test/coverage gate before handover.

Handoff: Give the Project Coordinator the test command/output and give Member 5 any contract failures as bugs.

Sprint 1 - Member 4

Student overflow plus shared responsive and accessible interface

REQUIRED BEFORE HANDOVER

See page 7 (run tests), page 8 (check the feature locally), page 9 (Actions runner rule) and page 12 (Global Definition of Done).

SPRINT 1 BASELINE NOTE

This Member area is already part of the integrated Sprint 1 baseline. Use it to verify/repair the current behavior; do not recreate a second implementation of the same foundation.

S1-M4-1 | Student overflow protected UI shell (Sprint 1 mock boundary)

Branch: s1/m4/student-overflow | Reviewer: Member 3 | 4 points | Integrates to: Gitea main

Depends on: Role routing from S1-M1-2

Goal: As a student, I want a clear protected overflow list so that I can understand unclaimed work while the real Student API is completed later.

Acceptance checks

- Only STUDENT users can open the page.
- Open, closed, empty and failed-load mock states are clear.
- The browser uses the agreed mock/API boundary and never talks directly to Neon.

Member tasks

- ☐ Maintain the Student overflow route/list/cards.
- ☐ Preserve open/closed/empty/error states and phone layout.
- ☐ Keep the mock contract stable until Sprint 2 connects the real backend endpoint.

Handoff: Give Member 5 and Member 6 the exact work-item fields needed by the UI.

S1-M4-2 | Shared role navigation and status components

Branch: s1/m4/shared-navigation | Reviewer: Member 3 | 4 points | Integrates to: Gitea main

Depends on: Role map from S1-M1-2

Goal: As any Tutor Mx user, I want consistent role navigation and status messages so that the app feels like one product.

Acceptance checks

- Only navigation permitted for the signed-in role is shown.
- Loading, empty, error, success and confirmation states use consistent accessible components.
- Sign out is reachable from Organiser, Tutor and Student workspaces.

Member tasks

- ☐ Maintain shared navigation/status components.
- ☐ Keep visible focus and skip-to-content behavior.
- ☐ Test role visibility and responsive navigation.

Handoff: Members 2 and 3 consume these components; Member 1 supplies the authoritative role list.

S1-M4-3 | Student volunteer confirmation and shared UI tests (Sprint 1 mock boundary)

Branch: s1/m4/volunteer-confirmation | Reviewer: Member 3 | 4 points | Integrates to: Gitea main

Depends on: S1-M4-1 and S1-M4-2

Goal: As a student, I want a clear confirm/cancel volunteer interaction so that duplicate or accidental claims are prevented.

Acceptance checks

- Confirmation repeats the selected work item and Cancel sends no request.
- Pending state blocks duplicate submission and success/failure is explained.
- Sprint 1 behavior remains on the agreed mock boundary until Sprint 2 real endpoints are ready.

Member tasks

- ☐ Maintain confirm/cancel and pending states.
- ☐ Test duplicate, already-claimed and failure paths.
- ☐ Hand the real request/error expectations to Sprint 2 backend work.

Handoff: Member 5 receives expected duplicate/error responses; the Project Coordinator receives test evidence.

Sprint 1 - Member 5

Handwritten Express API, validation, business rules, security and API tests

REQUIRED BEFORE HANDOVER

See page 7 (run tests), page 8 (check the feature locally), page 9 (Actions runner rule) and page 12 (Global Definition of Done).

SPRINT 1 BASELINE NOTE

This Member area is already part of the integrated Sprint 1 baseline. Use it to verify/repair the current behavior; do not recreate a second implementation of the same foundation.

S1-M5-1 | Separate handwritten Express backend and health endpoint

Branch: s1/m5/backend-foundation | Reviewer: Member 6 | 4 points | Integrates to: Gitea main

Depends on: Chosen backend runtime and repository structure

Goal: As the team, we want a separately runnable handwritten API so that browser code never depends on direct database access.

Acceptance checks

- Backend starts independently and GET /health returns a documented safe response.
- Unhandled routes/errors use the shared safe error shape.
- Environment validation/logging never exposes secrets.

Member tasks

- ☐ Maintain the Express server, central error handling and health/readiness checks.
- ☐ Keep backend lint/test/build scripts working from the repository root.
- ☐ Cover health and error behavior with tests.

Handoff: Give Member 6 the environment and database-connection requirements; give all frontend Members the base URL.

S1-M5-2 | Auth middleware and core profile/course/tutor/tutor-workflow routes

Branch: s1/m5/core-api-contracts | Reviewer: Member 6 | 4 points | Integrates to: Gitea main

Depends on: S1-M5-1, auth contract from Member 1 and schema from

Goal: As the application, we want protected handwritten endpoints so that all role data reaches React through one authorised API.

Acceptance checks

- Bearer verification and role/ownership middleware protect core routes.
- Course/tutor organiser routes and own-tutor dashboard/forms use Prisma on the backend only.
- Tutor workflow routes derive identity from the token, not a browser tutor ID.

Member tasks

- ☐ Maintain the current-profile, course, tutor and tutor-workflow routes.
- ☐ Keep success/error JSON and status codes consistent.
- ☐ Add success, validation, 401, 403, 404 and conflict tests when contracts change.

Handoff: Members 1-4 use the published route/payload examples; Member 6 aligns migrations and seed data.

S1-M5-3 | Three allocation-rule functions and boundary tests

Branch: s1/m5/allocation-rules | Reviewer: Member 6 | 4 points | Integrates to: Gitea main

Depends on: Threshold decisions from client/coordinator and Member 6 data

Goal: As an organiser, I want server-side mark, clash and weekly-hour rules so that invalid allocations cannot be bypassed by the UI.

Acceptance checks

- Mark threshold and exact weekly limit follow the agreed inclusive boundaries.
- Touching time ranges are not a clash; true overlaps are.
- Missing/malformed data returns controlled unknown/failure rather than silent approval.

Member tasks

- ☐ Maintain pure rule functions and stable reason codes.
- ☐ Keep boundary tests for threshold, overlap and exact/exceeded hours.
- ☐ Ensure UI consumes server results instead of duplicating business logic.

Handoff: Member 2 maps reason codes to UI text; Member 6 provides candidate/availability/hour data.

Sprint 1 - Member 6

Neon PostgreSQL, Prisma migrations, external integration, data engine, CI/deployment

REQUIRED BEFORE HANDOVER

See page 7 (run tests), page 8 (check the feature locally), page 9 (Actions runner rule) and page 12 (Global Definition of Done).

SPRINT 1 BASELINE NOTE

This Member area is already part of the integrated Sprint 1 baseline. Use it to verify/repair the current behavior; do not recreate a second implementation of the same foundation.

S1-M6-1 | Neon PostgreSQL schema, Prisma migrations and safe seed data

Branch: s | Reviewer: assigned reviewer | 4 points | Integrates to: Gitea main

Depends on: Field lists from Members 1-5

Goal: As the team, we want repeatable migrations and safe demo data so that every environment has the same data rules.

Acceptance checks

- Migrations create the required role/profile/course/mark/availability/hour/allocation structures from a clean database.
- Constraints reject invalid relationships and values.
- Seed data is invented/non-personal and no DATABASE_URL is committed.

Member tasks

- [] Maintain versioned Prisma migrations and seed data.
- [] Run schema/database verification after changes.
- [] Coordinate new fields with API and UI owners before migration changes land.

Handoff: Member 5 consumes the backend database connection; Members 2-4 receive stable demo identifiers and fields.

S1-M6-2 | Gitea integration checks, public deployment and shared Sprint 1 URL

Branch: s1/m6/ci-codecov | Reviewer: Member 5 | 4 points | Integrates to: Gitea main

Depends on: Test scripts from Members 1-5 and repository admin access

Goal: As the team, we want one tested Gitea main plus a public demo so that everyone works from the same source and can demonstrate the integrated system.

Acceptance checks

- Gitea main is the official shared source; members work on branches and the Integration Lead integrates approved work.
- The public frontend loads at tutor-mx.pages.dev and calls the hosted Express API rather than Neon directly.
- Local tests remain required even when the lecturer-managed Actions runner is queued/offline.

Member tasks

- [] Maintain Gitea workflow files and honest CI/runner evidence.
- [] Sync approved Gitea main to the GitHub deployment mirror when releasing.
- [] Build the frontend and manually upload the fresh frontend/dist folder to Cloudflare Production when required.

Handoff: All Members use the same test script names; the Integration Lead uses these checks as the merge gate.

S1-M6-3 | South African public-holiday API spike and fallback contract

Branch: s1/m6/external-api-spike | Reviewer: Member 5 | 4 points | Integrates to: Gitea main

Depends on: Client/coordinator chooses a relevant service and confirms

Goal: As Tutor Mx, we want a safe external public-holiday source to support scheduling without making the core system depend on that service.

Acceptance checks

- The backend-only spike returns a safe subset of South African holiday data.
- Timeout/unavailable/invalid responses return a controlled fallback.
- No user identity, token or database secret is sent to the external service.

Member tasks

- [] Maintain the Nager.Date holiday adapter/spike contract.
- [] Keep mocked success/timeout/invalid-response tests deterministic.
- [] Carry production route/display integration into later sprints rather than redoing the spike.

Handoff: Member 5 will integrate the adapter into the handwritten API; Member 4 will later display the safe response/fallback.

Sprint 1: integration and review gate

- [] Every Member starts from the latest Gitea main and hands over only their own tested branch.
- [] Assigned reviewer reproduces the issue-specific checks and checks page 7/8 evidence.
- [] Integration Lead integrates approved branches into main one at a time and reruns the affected regression/full gate.
- [] If Gitea Actions is unavailable because the lecturer runner is offline, record the blocker and local evidence honestly; do not mark it green.
- [] After integration, run the relevant three-role/public-environment smoke test.
- [] Update the GitHub deployment mirror and manual Cloudflare Production artifact only from approved Gitea main when a release is required.
- [] Issue is Done only when integrated behavior passes; code merely existing on a Member branch is not Done.

SPRINT 1 SPECIFIC RELEASE CHECK

Verify new Student/Tutor/Organiser account creation -> Auth0 -> automatic onboarding -> correct role dashboard. Verify Organiser Tutor Management contains no Auth0-subject input.

Sprint 2: plan, order and handovers

26 August-15 September 2026 | Complete Basic journeys without rebuilding Sprint 1 foundations

DO NOT REPEAT EARLIER SPRINTS

Sprint 2 starts from the latest approved Gitea main. Do not recreate Auth0 onboarding, core role routing, existing schema/API/UI foundations or manual tutor-ID workflows. Add only the new scope and hardening listed below.

- Finish the remaining Basic end-to-end gaps: Student real API, allocation writes, approvals/statuses and hardening.
- Reuse the working automatic account linking and registered-tutor management from Sprint 1.
- Run representative-user and three-role regression before release.

Member	Sprint ownership
Member 1	Authentication, role security, notifications/search and shared release checks
Member 2	Organiser interface, allocation board, approvals, reports and scenarios
Member 3	Tutor dashboard, availability, timesheets, excuses, imports and swaps
Member 4	Student overflow plus shared responsive and accessible interface
Member 5	Handwritten Express API, validation, business rules, security and API tests
Member 6	Neon PostgreSQL, Prisma migrations, external integration, data engine, CI/deployment
EVERY MEMBER Before handover: use page 7 for automated tests, page 8 for local browser verification, page 9 for the lecturer-runner rule, and page 12 for the Global Definition of Done.	

Sprint 2 - Member 1

Authentication, role security, notifications/search and shared release checks

REQUIRED BEFORE HANDOVER

See page 7 (run tests), page 8 (check the feature locally), page 9 (Actions runner rule) and page 12 (Global Definition of Done).

CARRY FORWARD

Start from the latest Gitea main and reuse the working Sprint 1 implementation. The three cards below are the new work for this sprint.

S2-M1-1 | Auth lifecycle hardening and email-verification recovery

Branch: s2/m1/account-lifecycle | Reviewer: Member 3 | 4 points | Integrates to: Gitea main

Depends on: Sprint 1 auth foundation and deployed backend

Goal: As a Tutor Mx user, I want edge cases around verification, reset, sign-out and deletion handled reliably without repeating the working Sprint 1 onboarding flow.

Acceptance checks

- A linked existing account never returns to role choice after normal sign-in.
- An unverified email gets a clear retry/recovery path; add resend-verification support if the agreed Auth0 approach allows it.
- Reset/deletion/network failure cannot leave the app in a false signed-in or half-deleted state.

Member tasks

- [] Keep Sprint 1 self-service onboarding unchanged unless a defect is proven.
- [] Harden verification/retry, reset, deletion and post-action session cleanup.
- [] Add integration tests for failure/recovery paths.

Handoff: Members 2-4 receive a stable signed-in session/role API; Coordinator receives acceptance evidence.

S2-M1-2 | Role and ownership protection across the completed Basic API

Branch: s2/m1/role-protection | Reviewer: Member 3 | 4 points | Integrates to: Gitea main

Depends on: S2-M1-1 and Member 5 Basic endpoints

Goal: As the school, we want every real Basic endpoint checked for role and ownership so that hidden navigation can never be bypassed.

Acceptance checks

- Organiser-only writes return 403 for Tutor/Student callers.
- Own-data routes do not reveal another Tutor/Student when identifiers are manipulated.
- Frontend route guards and backend authorisation remain consistent.

Member tasks

- [] Create an endpoint-to-role/ownership matrix for the now-real Basic routes.
- [] Add 401/403/cross-user tests for new Sprint 2 endpoints.
- [] Fix permission defects without moving security logic into React.

Handoff: Member 5 receives failing endpoint cases; Members 2-4 use the shared permission/error behavior.

S2-M1-3 | Cross-role security smoke suite and assigned fixes

Branch: s2/m1/security-smoke | Reviewer: Member 3 | 4 points | Integrates to: Gitea main

Depends on: All Sprint 2 Basic journeys integrated on Gitea main

Goal: As the Integration Lead, we want a repeatable three-role smoke suite so that integration cannot break access or expose another role's data.

Acceptance checks

- Separate Student/Tutor/Organiser test accounts land only in their expected workspace.
- Critical endpoints show consistent 401/403 behavior for missing, expired and wrong-role tokens.
- Any severity-one/two security defect blocks release until fixed or formally recorded as a blocker.

Member tasks

- [] Create/run the cross-role smoke sequence against the shared environment.
- [] Open reproducible bugs for failures and fix Member 1-owned defects.
- [] Attach local test evidence even when the Gitea runner is unavailable.

Handoff: Integration Lead runs this before the Integration Lead release decision; Coordinator links results into testing evidence.

Sprint 2 - Member 2

Organiser interface, allocation board, approvals, reports and scenarios

REQUIRED BEFORE HANDOVER

See page 7 (run tests), page 8 (check the feature locally), page 9 (Actions runner rule) and page 12 (Global Definition of Done).

CARRY FORWARD

Start from the latest Gitea main and reuse the working Sprint 1 implementation. The three cards below are the new work for this sprint.

S2-M2-1 | Organiser course/tutor management hardening - registered tutors only

Branch: s2/m2/organiser-crud | Reviewer: Member 5 | 4 points | Integrates to: Gitea main

Depends on: Member 5 organiser endpoints and Member 6 Basic schema

Goal: As an organiser, I want the working Sprint 1 management screens hardened around real registered Tutor accounts instead of rebuilding tutor creation.

Acceptance checks

- Tutor search/list comes from self-onboarded Tutor profiles and never asks for Auth0 subject/User ID.
- Edit/validation/refresh behavior stays correct after API failures.
- Course CRUD and tutor details/hour-limit/marks/busy-time edits persist and refresh cleanly.

Member tasks

- ☐ Do not rebuild Sprint 1 Tutor Management or add a manual tutor-create form.
- ☐ Fix validation, empty/error states, responsive layout and stale-data issues found in shared testing.
- ☐ Add integration tests around real course/tutor edit flows.

Handoff: Member 5 fixes contract mismatches; Member 6 validates data constraints.

S2-M2-2 | Live allocation create, edit and remove with server checks

Branch: s2/m2/allocation-board-basic | Reviewer: Member 5 | 4 points | Integrates to: Gitea main

Depends on: S2-M2-1, Member 5 allocation endpoints and Member 6 queries

Goal: As an organiser, I want the Sprint 1 allocation placeholder replaced by real save/edit/remove behavior while keeping all rule checks visible.

Acceptance checks

- A valid candidate can be saved through a real allocation-write endpoint.
- Failed mark/clash/hour rules are rejected server-side and remain visible in the UI.
- Editing/removing an allocation updates hours and assignments without manual database edits.

Member tasks

- ☐ Connect real allocation create/edit/remove endpoints.
- ☐ Refresh candidate hours and assignment data after every successful change.
- ☐ Test each failed rule plus valid create/edit/remove.

Handoff: Member 5 enforces rules on server; Member 3 sees resulting tutor obligations.

S2-M2-3 | Organiser timesheet and overflow approvals with tests

Branch: s2/m2/organiser-approvals | Reviewer: Member 5 | 4 points | Integrates to: Gitea main

Depends on: Member 3 timesheets, Member 4 volunteer requests and

Goal: As an organiser, I want to approve or return tutor timesheets and approve or reject student volunteer claims so that Basic workflows can complete end to end.

Acceptance checks

- Pending timesheets and claims show the information needed for a decision.
- Approve/reject/return actions persist a clear status and update the responsible user view.
- Duplicate or stale decisions are rejected safely.

Member tasks

- ☐ Build approval queues/views on the Organiser workspace.
- ☐ Connect to the Sprint 2 workflow endpoints and confirmation states.
- ☐ Test valid, rejected, stale and API-failure cases.

Handoff: Members 3 and 4 consume the resulting statuses; Coordinator receives demo-ready accounts/data.

Sprint 2 - Member 3

Tutor dashboard, availability, timesheets, excuses, imports and swaps

REQUIRED BEFORE HANDOVER

See page 7 (run tests), page 8 (check the feature locally), page 9 (Actions runner rule) and page 12 (Global Definition of Done).

CARRY FORWARD

Start from the latest Gitea main and reuse the working Sprint 1 implementation. The three cards below are the new work for this sprint.

S2-M3-1 | Tutor dashboard hardening and live assignment refresh

Branch: s2/m3/tutor-dashboard-api | Reviewer: Member 1 | 4 points | Integrates to: Gitea main

Depends on: Member 5 own-dashboard endpoint and Member 2 allocations

Goal: As a tutor, I want the already-live Sprint 1 dashboard to stay accurate when allocations change and when my weekly limit is unset or updated.

Acceptance checks

- The dashboard shows only the signed-in tutor and refreshes after allocation changes.
- No-work and no-hour-limit states remain usable.
- Failed refresh preserves the last safe state and offers retry.

Member tasks

- [] Do not replace the working own-tutor API with a new tutor-ID flow.
- [] Add refresh/status handling for allocation changes.
- [] Expand dashboard tests for changed assignments, unset limits and retry.

Handoff: Member 2 confirms allocation changes; Member 5 fixes data/permission contract issues.

S2-M3-2 | Complete timesheet submission workflow from existing work logs

Branch: s2/m3/timesheet-basic | Reviewer: Member 1 | 4 points | Integrates to: Gitea main

Depends on: S2-M3-1 and Member 5 work-log/timesheet endpoints

Goal: As a tutor, I want my existing work logs to become a submit/review timesheet workflow so that organisers can approve actual worked hours.

Acceptance checks

- Draft work logs can be grouped/submitted into the agreed timesheet state.
- Invalid hours, duplicate submissions and wrong-session ownership are rejected.
- Tutor sees pending/approved/returned status after organiser action.

Member tasks

- [] Build submit/review-status UI on top of Sprint 1 work-log creation.
- [] Connect real timesheet submit/status endpoints.
- [] Test draft, submit, duplicate, returned and approved cases.

Handoff: Member 2 consumes submitted timesheets; Member 6 validates totals/status constraints.

S2-M3-3 | Excuse status workflow and Tutor Basic regression

Branch: s2/m3/excuse-regression | Reviewer: Member 1 | 4 points | Integrates to: Gitea main

Depends on: S2-M3-1, S2-M3-2 and Member 5 excuse endpoint

Goal: As a tutor, I want the Sprint 1 excuse submission to show its later decision/status and remain reliable with the rest of the Basic journey.

Acceptance checks

- Submitted excuses show pending and later organiser outcome.
- Past/unowned sessions cannot be excused.
- Tutor regression covers dashboard, availability, work log, timesheet and excuse paths together.

Member tasks

- [] Add excuse status/history display rather than recreating submission.
- [] Connect organiser decision outcome once the endpoint exists.
- [] Run the full tutor Basic regression and fix reproducible defects.

Handoff: Coordinator receives a complete tutor demo script result; Member 2 sees excuse/availability impact.

Sprint 2 - Member 4

Student overflow plus shared responsive and accessible interface

REQUIRED BEFORE HANDOVER

See page 7 (run tests), page 8 (check the feature locally), page 9 (Actions runner rule) and page 12 (Global Definition of Done).

CARRY FORWARD

Start from the latest Gitea main and reuse the working Sprint 1 implementation. The three cards below are the new work for this sprint.

S2-M4-1 | Connect Student overflow to real open-work API

Branch: s2/m4/overflow-api | Reviewer: Member 6 | 4 points | Integrates to: Gitea main

Depends on: Member 5 overflow endpoint and Member 6 schema

Goal: As a student, I want the Sprint 1 overflow shell to load real open work so that the mock boundary is removed without changing the established UI behavior.

Acceptance checks

- Only eligible/open work is returned for the signed-in Student.
- Closed/claimed work cannot be volunteered for.
- Loading, empty, error and retry states remain clear.

Member tasks

- ☐ Replace the Sprint 1 mock with the real student overflow endpoint.
- ☐ Keep the existing responsive list/cards and state components.
- ☐ Add integration tests for open, closed, empty and failure cases.

Handoff: Member 5 fixes response/permission issues; Member 6 supplies reproducible open/closed seed cases.

S2-M4-2 | Volunteer request and outcome statuses

Branch: s2/m4/volunteer-basic | Reviewer: Member 6 | 4 points | Integrates to: Gitea main

Depends on: S2-M4-1 and Member 5 volunteer endpoint

Goal: As a student, I want my confirmed volunteer request stored and its outcome shown so that I know whether I received the work.

Acceptance checks

- Confirm creates one request and duplicate clicks/claims are controlled.
- Pending/approved/rejected/already-claimed statuses are visible.
- Another Student cannot view or mutate the request.

Member tasks

- ☐ Connect the Sprint 1 confirmation UI to the real volunteer endpoint.
- ☐ Show persisted request/outcome states.
- ☐ Test duplicate, race, rejected and permission cases.

Handoff: Member 2 consumes pending claims; Member 5/6 enforce uniqueness and atomic outcome.

S2-M4-3 | External-result UI, accessibility and Student Basic regression

Branch: s2/m4/shared-quality | Reviewer: Member 6 | 4 points | Integrates to: Gitea main

Depends on: Member 6 external adapter and all Basic shared UI

Goal: As a user, I want the approved external holiday result/fallback and Student journey presented accessibly without weakening the working shared UI.

Acceptance checks

- External information is clearly supporting information and a safe fallback appears when unavailable.
- Student primary actions remain keyboard/mobile accessible.
- Student regression covers real overflow and volunteer flows.

Member tasks

- ☐ Render only the safe external API response/fallback from the backend.
- ☐ Run accessibility/responsive checks on Student and shared components.
- ☐ Fix Member 4-owned Basic regressions found in representative-user testing.

Handoff: Coordinator receives accessibility/test evidence; Member 6 receives any adapter failure cases.

Sprint 2 - Member 5

Handwritten Express API, validation, business rules, security and API tests

REQUIRED BEFORE HANDOVER

See page 7 (run tests), page 8 (check the feature locally), page 9 (Actions runner rule) and page 12 (Global Definition of Done).

CARRY FORWARD

Start from the latest Gitea main and reuse the working Sprint 1 implementation. The three cards below are the new work for this sprint.

S2-M5-1 | Allocation-write and organiser-approval API completion

Branch: s2/m5/organiser-api | Reviewer: Member 2 | 4 points | Integrates to: Gitea main

Depends on: Member 6 Basic schema and Member 1 role matrix

Goal: As the application, we want real allocation mutations and organiser approval routes so Sprint 2 can complete workflows without redoing Sprint 1 course/tutor endpoints.

Acceptance checks

- Allocation create/edit/remove rechecks mark, clash and hours on the server inside safe database operations.
- Timesheet/claim organiser decisions enforce allowed state transitions.
- Legacy POST /api/tutors is not required by the normal self-service Tutor UI.

Member tasks

- [] Implement allocation mutation routes and transaction/service logic.
- [] Implement organiser timesheet/claim decision routes.
- [] Add validation, permission, conflict and state-transition tests.

Handoff: Member 2 connects the board; Member 6 tunes queries/constraints.

S2-M5-2 | Student overflow/volunteer and timesheet-status API

Branch: s2/m5/tutor-student-api | Reviewer: Member 2 | 4 points | Integrates to: Gitea main

Depends on: Member 6 Basic workflow schema and Member 1 ownership rules

Goal: As the frontend, we want the missing Student and workflow endpoints so the remaining Sprint 1 mocks/status gaps can become real.

Acceptance checks

- Student open-work and volunteer routes are role/ownership safe.
- Timesheet submit/status and excuse decision/status contracts are stable.
- Duplicate/race/state conflicts use safe 409-style responses.

Member tasks

- [] Implement Student overflow/volunteer endpoints.
- [] Complete timesheet/excuse status routes needed by Tutor/Organiser UI.
- [] Add success, validation, 401/403/404/409 tests.

Handoff: Members 3 and 4 connect their screens; Member 6 enforces database uniqueness/transitions.

S2-M5-3 | API contract, deployment and coverage hardening

Branch: s2/m5/api-quality | Reviewer: Member 2 | 4 points | Integrates to: Gitea main

Depends on: S2-M5-1, S2-M5-2 and CI from Member 6

Goal: As the team, we want the expanded Sprint 2 API to keep the existing safe error contract and remain deployable/testable as one service.

Acceptance checks

- New routes use the established { error: { code, message } } shape and do not leak stack traces/secrets.
- Hosted health/smoke behavior stays stable after deployment.
- Coverage includes critical success, permission, validation and conflict branches.

Member tasks

- [] Standardise only new/changed routes; do not rewrite already-stable Sprint 1 contracts unnecessarily.
- [] Run backend build/smoke/coverage and fix real regressions.
- [] Provide deployment and API evidence to the Integration Lead.

Handoff: Integration Lead uses health/tests/Codecov as merge evidence; Coordinator uses route examples for API docs.

Sprint 2 - Member 6

Neon PostgreSQL, Prisma migrations, external integration, data engine, CI/deployment

REQUIRED BEFORE HANDOVER

See page 7 (run tests), page 8 (check the feature locally), page 9 (Actions runner rule) and page 12 (Global Definition of Done).

CARRY FORWARD

Start from the latest Gitea main and reuse the working Sprint 1 implementation. The three cards below are the new work for this sprint.

S2-M6-1 | Sprint 2 migrations for workflow state and allocation writes

Branch: s2/m6/basic-workflow-schema | Reviewer: Member 4 | 4 points | Integrates to: Gitea main

Depends on: Final Basic payloads from Members 2-5

Goal: As the team, we want only the new Sprint 2 data/state additions migrated so the working Sprint 1 schema is extended rather than recreated.

Acceptance checks

- New allocation mutation, timesheet/approval, excuse-status and volunteer-request data is represented with safe constraints.
- Migrations apply from the existing Sprint 1 schema and from a clean database.
- No manual production table edits are required.

Member tasks

- ☐ Add minimal versioned migrations for new Sprint 2 states/relations.
- ☐ Update seed data for approval and race/conflict cases.
- ☐ Run db verification and coordinate schema changes with API owners.

Handoff: Member 5 connects transactions/queries; Members 2-4 use stable demo data.

S2-M6-2 | Production public-holiday adapter with graceful fallback

Branch: s2/m6/external-api-adapter | Reviewer: Member 4 | 4 points | Integrates to: Gitea main

Depends on: Approved Sprint 1 external service and Member 5 backend

Goal: As Tutor Mx, we want the Sprint 1 holiday spike exposed safely through the backend with caching/fallback so the UI can use it without repeating the research spike.

Acceptance checks

- Only the backend calls the external service and returns the approved safe fields.
- Timeout/unavailable/invalid-response uses the established fallback.
- The integration does not block core Tutor Mx workflows.

Member tasks

- ☐ Turn the validated Sprint 1 spike into a production backend adapter/route.
- ☐ Add sensible caching and deterministic mocked tests.
- ☐ Document the fallback and any service limits.

Handoff: Member 5 exposes the adapter through handwritten API; Member 4 renders success/fallback.

S2-M6-3 | Allocation data performance, deployment smoke and Codecov reliability

Branch: s2/m6/data-delivery-quality | Reviewer: Member 4 | 4 points | Integrates to: Gitea main

Depends on: S2-M6-1, Member 5 API and Gitea Actions

Goal: As the Integration Lead, we want the completed Basic data flow and release checks reliable enough for repeated integration.

Acceptance checks

- Allocation candidates/writes and dashboard refresh use efficient queries with correct results.
- Clean setup/build/smoke works against the shared environment.
- When the lecturer runner is available, Gitea Actions/coverage run; when unavailable, the blocker plus local evidence is recorded.

Member tasks

- ☐ Profile/fix obvious allocation query issues and database constraints.
- ☐ Run clean migration/seed/check/smoke before release.
- ☐ Maintain Gitea CI/coverage config without inventing green runner evidence.

Handoff: Integration Lead uses this as Sprint 2 merge gate; Coordinator receives run links and screenshots.

Sprint 2: integration and review gate

- [] Every Member starts from the latest Gitea main and hands over only their own tested branch.
- [] Assigned reviewer reproduces the issue-specific checks and checks page 7/8 evidence.
- [] Integration Lead integrates approved branches into main one at a time and reruns the affected regression/full gate.
- [] If Gitea Actions is unavailable because the lecturer runner is offline, record the blocker and local evidence honestly; do not mark it green.
- [] After integration, run the relevant three-role/public-environment smoke test.
- [] Update the GitHub deployment mirror and manual Cloudflare Production artifact only from approved Gitea main when a release is required.
- [] Issue is Done only when integrated behavior passes; code merely existing on a Member branch is not Done.

SPRINT 2 SPECIFIC RELEASE CHECK

Verify real Student overflow/volunteer, allocation create/edit/remove and organiser approval/status journeys end to end.
Confirm no Sprint 1 manual-ID flow was reintroduced.

Sprint 3: plan, order and handovers

16-29 September 2026 | *Intermediate features, feedback fixes and near-complete quality*

DO NOT REPEAT EARLIER SPRINTS

Sprint 3 starts from the latest approved Gitea main. Do not recreate Auth0 onboarding, core role routing, existing schema/API/UI foundations or manual tutor-ID workflows. Add only the new scope and hardening listed below.

- Add Intermediate ranking/bulk/shortage/override/import/report/notification/search workflows on top of Basic.
- “Tutor onboarding” in Sprint 3 means first-use guidance, not Auth0 role/Profile creation.
- Expand performance, accessibility, security and feedback regression.

Member	Sprint ownership
Member 1	Authentication, role security, notifications/search and shared release checks
Member 2	Organiser interface, allocation board, approvals, reports and scenarios
Member 3	Tutor dashboard, availability, timesheets, excuses, imports and swaps
Member 4	Student overflow plus shared responsive and accessible interface
Member 5	Handwritten Express API, validation, business rules, security and API tests
Member 6	Neon PostgreSQL, Prisma migrations, external integration, data engine, CI/deployment
EVERY MEMBER Before handover: use page 7 for automated tests, page 8 for local browser verification, page 9 for the lecturer-runner rule, and page 12 for the Global Definition of Done.	

Sprint 3 - Member 1

Authentication, role security, notifications/search and shared release checks

REQUIRED BEFORE HANDOVER

See page 7 (run tests), page 8 (check the feature locally), page 9 (Actions runner rule) and page 12 (Global Definition of Done).

CARRY FORWARD

Start from the latest Gitea main and reuse the working Sprints 1-2 implementation. The three cards below are the new work for this sprint.

S3-M1-1 | Notification inbox and due reminders

Branch: s3/m1/notifications | Reviewer: Member 4 | 4 points | Integrates to: Gitea main

Depends on: Member 6 notification data/job and Member 5 endpoints

Goal: As an affected user, I want assignment changes and due items to reach me so that I do not miss important Tutor Mx work.

Acceptance checks

- Given an assignment/status change affecting a user, when it occurs, then one relevant notification appears for that user.
- Given a due timesheet/approval, when the reminder rule runs, then the person currently responsible is notified.
- Given an unrelated user, when notifications load, then they cannot see another user's messages.

Member tasks

- [] Build notification inbox/list and unread/read state.
- [] Add deep links for permitted assignment/timesheet/claim items.
- [] Connect to Member 5 endpoints and Member 6 reminder data.

Handoff: Members 2-4 reuse notification components; Member 5/6 receive duplication/permission failures.

S3-M1-2 | Global search and first-use tutor guidance

Branch: s3/m1/search-onboarding | Reviewer: Member 4 | 4 points | Integrates to: Gitea main

Depends on: Member 5 search endpoint and role navigation

Goal: As a user, I want role-safe search and contextual first-use guidance so I can find work and understand missing setup without recreating Auth0 role onboarding.

Acceptance checks

- Search returns only authorised result types and never leaks hidden records.
- A Tutor with missing hour limit/assignments gets helpful next-step guidance, not a second role-selection/account-creation flow.
- Guidance can be dismissed/resumed without changing the user's established role.

Member tasks

- [] Build shared search and grouped result states.
- [] Add first-use guidance around profile/hour-limit/assignment setup only.
- [] Do not reassign Auth0 roles or recreate Neon Profiles from this feature.

Handoff: Member 5 supplies role-filtered search; Member 3 links onboarding to tutor screens.

S3-M1-3 | Shared performance, accessibility and feedback fixes

Branch: s3/m1/shared-quality-fixes | Reviewer: Member 4 | 4 points | Integrates to: Gitea main

Depends on: Coordinator's prioritised user-feedback issues and integrated

Goal: As the team, we want high-priority shared problems fixed and measured so that the near-complete product is usable, secure and fast enough.

Acceptance checks

- Given prioritised feedback assigned to Member 1, when fixed, then the linked acceptance test passes and the issue references the PR.
- Given account/search/notification critical paths, when measured, then they meet the team's recorded response target or a justified limitation is logged.
- Given keyboard/focus/labels, when shared paths are checked, then no critical failure remains.

Member tasks

- [] Implement the highest-priority shared feedback fixes assigned by the Coordinator.
- [] Measure and improve one slow shared path.
- [] Run focused accessibility checks and fix critical findings.

Handoff: Coordinator closes feedback items only after evidence; Integration Lead checks regression/Codecov.

Sprint 3 - Member 2

Organiser interface, allocation board, approvals, reports and scenarios

REQUIRED BEFORE HANDOVER

See page 7 (run tests), page 8 (check the feature locally), page 9 (Actions runner rule) and page 12 (Global Definition of Done).

CARRY FORWARD

Start from the latest Gitea main and reuse the working Sprints 1-2 implementation. The three cards below are the new work for this sprint.

S3-M2-1 | Ranked candidates and shortage visibility

Branch: s3/m2/ranked-candidates | Reviewer: Member 6 | 4 points | Integrates to: Gitea main

Depends on: Member 5 ranking endpoint and Member 6 ranking/query logic

Goal: As an organiser, I want eligible tutors ranked and shortages visible so that the system does more of the allocation legwork.

Acceptance checks

- Given a course/session, when candidates load, then eligible tutors are ordered by the documented ranking inputs.
- Given an ineligible tutor, when listed/excluded, then the organiser can see the reason.
- Given unmet required work, when the board renders, then the shortage amount/location is visible.

Member tasks

- [] Connect ranked candidate results to the board.
- [] Show concise reasons/inputs without pretending the score is objective truth.
- [] Add shortage indicators at course/session level.

Handoff: Member 6 owns algorithm/data correctness; Member 5 owns response stability.

S3-M2-2 | Bulk fill and reasoned organiser override

Branch: s3/m2/bulk-override | Reviewer: Member 6 | 4 points | Integrates to: Gitea main

Depends on: S3-M2-1 and Member 5 bulk/override endpoints

Goal: As an organiser, I want obvious assignments filled in bulk and exceptions overridden with a recorded reason so that repetitive allocation is faster but accountable.

Acceptance checks

- Given safe obvious assignments, when previewed in bulk, then proposed changes are listed before confirmation.
- Given confirmation, when bulk fill succeeds/fails partly, then each item has a clear result and no hidden partial state.
- Given an allowed override, when submitted, then a non-empty reason is required and shown in history.

Member tasks

- [] Build bulk preview, select/deselect and confirm UI.
- [] Handle full success, partial result and failure states.
- [] Build override action with required reason and warning.

Handoff: Member 5/6 define transaction and allowed-override rules; Coordinator receives audit evidence.

S3-M2-3 | Hours, budget and distribution reports with keyboard board use

Branch: s3/m2/organiser-reports | Reviewer: Member 6 | 4 points | Integrates to: Gitea main

Depends on: Member 5 report endpoints and Coordinator-approved budget

Goal: As an organiser, I want hours, spend and work distribution plus efficient keyboard actions so that I can monitor and operate a large allocation board.

Acceptance checks

- Given allocations/timesheets, when reports load, then hours per course, spend against budget and work distribution use documented calculations.
- Given no/partial data, when reports load, then totals state the limitation rather than inventing values.
- Given keyboard-only use, when common board actions run, then selection, validation and bulk actions can be completed.

Member tasks

- [] Build organiser report views/summary cards with clear units.
- [] Connect to backend calculations; do not recalculate authoritative totals differently in UI.
- [] Add keyboard shortcuts/actions with visible help and no conflict with browser basics.

Handoff: Member 5/6 provide calculations/load data; Coordinator records formula/target in docs.

Sprint 3 - Member 3

Tutor dashboard, availability, timesheets, excuses, imports and swaps

REQUIRED BEFORE HANDOVER

See page 7 (run tests), page 8 (check the feature locally), page 9 (Actions runner rule) and page 12 (Global Definition of Done).

CARRY FORWARD

Start from the latest Gitea main and reuse the working Sprints 1-2 implementation. The three cards below are the new work for this sprint.

S3-M3-1 | Timetable import, recurring exceptions and term calendar

Branch: s3/m3/timetable-import | Reviewer: Member 5 | 4 points | Integrates to: Gitea main

Depends on: Member 5 import/availability endpoints and Member 6 calendar

Goal: As a tutor, I want to import my timetable and add recurring exceptions so that availability is easier to provide accurately.

Acceptance checks

- Given a supported timetable file/format, when imported, then a preview shows parsed busy times before confirmation.
- Given invalid/partial input, when imported, then errors identify the affected rows without corrupting existing availability.
- Given recurring exception and term dates, when saved, then generated busy occurrences fall only within the term rule.

Member tasks

- ☐ Build upload/paste, preview, error and confirm UI.
- ☐ Add recurring-exception and term-calendar controls.
- ☐ Connect import/availability endpoints.

Handoff: Member 6 owns parser data/term rules; Member 2 sees resulting allocation clashes.

S3-M3-2 | Timesheet corrections, disputes and payroll export

Branch: s3/m3/timesheet-workflow | Reviewer: Member 5 | 4 points | Integrates to: Gitea main

Depends on: Member 5 workflow/export endpoints and Member 6 status

Goal: As a tutor and organiser, we want corrections/disputes and payroll-ready output so that timesheets move through a complete accountable workflow.

Acceptance checks

- Given a returned timesheet, when corrected/resubmitted, then previous values/reason remain traceable.
- Given a disputed decision, when a dispute is submitted, then status and reason route to the responsible organiser.
- Given an approved period, when export is requested, then the file contains the agreed fields/totals and excludes unapproved work.

Member tasks

- ☐ Build tutor correction and dispute views.
- ☐ Show status timeline/history clearly.
- ☐ Connect payroll export action and safe download state.

Handoff: Member 2 handles organiser side; Coordinator receives sample anonymised export evidence.

S3-M3-3 | Tutor first-use guidance, notifications and regression

Branch: s3/m3/tutor-intermediate-tests | Reviewer: Member 5 | 4 points | Integrates to: Gitea main

Depends on: S3-M1-1, S3-M1-2, S3-M3-1 and S3-M3-2

Goal: As a tutor, I want first-use guidance and notifications integrated with my mature Tutor journey without repeating account onboarding.

Acceptance checks

- Guidance reflects missing tutor configuration/work rather than asking the user to choose a role again.
- Relevant notifications deep-link only to permitted Tutor items.
- Regression covers imports, exceptions, timesheets, notices and existing Sprint 1/2 tutor flows.

Member tasks

- ☐ Integrate first-use guidance with the current Tutor dashboard.
- ☐ Connect notification components to Tutor-owned items.
- ☐ Run full Tutor regression and accessibility checks.

Handoff: Coordinator closes feedback issues; Integration Lead uses tutor regression as merge gate.

Sprint 3 - Member 4

Student overflow plus shared responsive and accessible interface

REQUIRED BEFORE HANDOVER

See page 7 (run tests), page 8 (check the feature locally), page 9 (Actions runner rule) and page 12 (Global Definition of Done).

CARRY FORWARD

Start from the latest Gitea main and reuse the working Sprints 1-2 implementation. The three cards below are the new work for this sprint.

S3-M4-1 | Overflow posting, claim and organiser-approval UI

Branch: s3/m4/overflow-workflow | Reviewer: Member 1 | 4 points | Integrates to: Gitea main

Depends on: Member 5 overflow workflow API and Member 6 schema

Goal: As organiser and student, we want overflow work posted, claimed and approved so that unassigned work reaches volunteers with control.

Acceptance checks

- Given an organiser, when valid overflow work is posted/closed, then student availability updates.
- Given a student, when an open item is claimed, then one pending claim is created.
- Given organiser approval/rejection, when decided, then only the approved outcome changes the assignment and the student is notified.

Member tasks

- [] Build organiser post/edit/close overflow UI.
- [] Complete student claim status/history UI.
- [] Build organiser claim approval/rejection UI or integrate Member 2 component.

Handoff: Member 5/6 enforce atomic claim rules; Member 1 supplies notification component.

S3-M4-2 | Shared search/notification states and bulk-action consistency

Branch: s3/m4/shared-intermediate-ui | Reviewer: Member 1 | 4 points | Integrates to: Gitea main

Depends on: Member 1 search/notifications and Member 2 bulk UI

Goal: As any user, I want new search, notification and bulk interactions to use consistent accessible states so that the larger system still feels predictable.

Acceptance checks

- Given loading/empty/error/success for new features, when shown, then shared components use consistent text, focus and announcements.
- Given unread notifications/search results, when keyboard navigation is used, then selection and return focus behave correctly.
- Given bulk confirmation, when cancel/partial failure occurs, then the final state is clear.

Member tasks

- [] Extend shared status/confirmation components for new Intermediate cases.
- [] Integrate accessible live-region/focus behavior where needed.
- [] Review and fix search, notification and bulk responsive issues.

Handoff: Members 1 and 2 adopt shared fixes; Coordinator records accessibility evidence.

S3-M4-3 | Representative-user fixes and app-wide accessibility regression

Branch: s3/m4/usability-regression | Reviewer: Member 1 | 4 points | Integrates to: Gitea main

Depends on: Coordinator's prioritised user-test issues and near-complete

Goal: As the team, we want representative-user problems and app-wide accessibility failures fixed so that Organiser, Tutor and Student journeys are usable.

Acceptance checks

- Given each representative role script, when run on the shared build, then critical tasks can be completed without coaching.
- Given assigned feedback findings, when fixed/re-tested, then evidence links issue, PR and passing step.
- Given keyboard/focus/label/status checks, when app-wide regression runs, then no critical failure remains.

Member tasks

- [] Implement high-priority user-feedback/accessibility fixes assigned to Member 4.
- [] Run a repeatable app-wide keyboard/focus/label checklist.
- [] Run responsive checks across the three role journeys.

Handoff: Coordinator owns sessions/notes; Member 4 implements technical fixes only.

Sprint 3 - Member 5

Handwritten Express API, validation, business rules, security and API tests

REQUIRED BEFORE HANDOVER

See page 7 (run tests), page 8 (check the feature locally), page 9 (Actions runner rule) and page 12 (Global Definition of Done).

CARRY FORWARD

Start from the latest Gitea main and reuse the working Sprints 1-2 implementation. The three cards below are the new work for this sprint.

S3-M5-1 | Ranking, bulk-fill and override API

Branch: s3/m5/allocation-intermediate-api | Reviewer: Member 3 | 4 points | Integrates to: Gitea main

Depends on: Member 6 ranking logic/schema and Basic allocation API

Goal: As the organiser UI, I want protected ranking, shortage, bulk and override endpoints so that Intermediate allocation behavior is consistent and auditable.

Acceptance checks

- Given a course/session, when ranking endpoint runs, then deterministic ordered candidates and reasons are returned.
- Given bulk preview/commit, when requested, then preview has no side effects and commit returns per-item results atomically or explicitly partially.
- Given allowed override, when a reason is supplied, then it is stored; missing reason is rejected.

Member tasks

- ☐ Implement ranking/shortage endpoints using Member 6 services.
- ☐ Implement bulk preview and commit contract.
- ☐ Implement allowed override with required reason/history.

Handoff: Member 2 consumes endpoints; Member 6 validates algorithm/transaction behavior.

S3-M5-2 | Intermediate workflow, search, report and notification API

Branch: s3/m5/intermediate-api | Reviewer: Member 3 | 4 points | Integrates to: Gitea main

Depends on: Member 6 Intermediate schema/jobs

Goal: As the expanded frontend, I want protected endpoints for timetable, timesheet workflow, overflow, search, reports and notifications so that Intermediate journeys stay behind the handwritten API.

Acceptance checks

- Given supported timetable/correction/dispute/overflow requests, when valid, then state transitions persist with history.
- Given search, when called, then results are role-filtered and paginated/limited.
- Given reports, when called, then hours/spend/distribution calculations are server-defined and testable.

Member tasks

- ☐ Implement/complete timetable and availability endpoints.
- ☐ Implement correction/dispute/export and overflow workflow endpoints.
- ☐ Implement role-safe search/report/notification endpoints.

Handoff: Members 1-4 connect screens; Member 6 tunes queries/data histories.

S3-M5-3 | API performance, security and Intermediate regression

Branch: s3/m5/api-intermediate-quality | Reviewer: Member 3 | 4 points | Integrates to: Gitea main

Depends on: All S3 API work on Gitea main and realistic Member 6 data

Goal: As the team, we want measured and regression-tested API behavior so that the near-complete system is stable and secure.

Acceptance checks

- Given realistic data, when critical board/search/report endpoints are measured, then they meet the recorded target or an issue/limit is documented.
- Given malformed/large/unauthorised requests, when sent, then validation, limits and permissions protect the service.
- Given full API regression, when CI runs, then all Basic and Intermediate endpoint suites pass and upload coverage.

Member tasks

- ☐ Add timing/load checks for critical API routes.
- ☐ Fix obvious unbounded/N+1/validation issues.
- ☐ Add security regression for roles, ownership, input sizes and logging.

Handoff: Integration Lead requires these checks; Coordinator records measured target/results.

Sprint 3 - Member 6

Neon PostgreSQL, Prisma migrations, external integration, data engine, CI/deployment

REQUIRED BEFORE HANDOVER

See page 7 (run tests), page 8 (check the feature locally), page 9 (Actions runner rule) and page 12 (Global Definition of Done).

CARRY FORWARD

Start from the latest Gitea main and reuse the working Sprints 1-2 implementation. The three cards below are the new work for this sprint.

S3-M6-1 | Intermediate schema, status history and report data

Branch: s3/m6/intermediate-schema | Reviewer: Member 2 | 4 points | Integrates to: Gitea main

Depends on: S3 feature payloads from Members 1-5

Goal: As the team, we want migrated data structures for Intermediate workflows so that timetable, corrections, disputes, claims, notifications and reports remain consistent.

Acceptance checks

- Given clean migrations, when run, then term calendar/import, recurring exception, status history, notification and budget/report fields exist.
- Given invalid transition/duplicate event, when stored, then constraints or transaction logic reject it.
- Given report queries, when run on known seed data, then hours/spend/distribution totals match expected fixtures.

Member tasks

- ☐ Add forward migrations for Intermediate entities/history.
- ☐ Add constraints/indexes for state, search and reporting queries.
- ☐ Create known fixtures for ranking/reports/notifications.

Handoff: Member 5 implements endpoints; Members 1-4 receive stable fixture cases.

S3-M6-2 | Ranking service, reminder jobs and allocation data engine

Branch: s3/m6/intermediate-engine | Reviewer: Member 2 | 4 points | Integrates to: Gitea main

Depends on: S3-M6-1 and Coordinator-approved ranking inputs

Goal: As an organiser, I want deterministic ranking, shortage calculation and reminders so that recommendations and due work are useful and explainable.

Acceptance checks

- Given candidate inputs, when ranking runs, then the order and contributing factors are deterministic and testable.
- Given shortages, when required versus assigned work is calculated, then the exact gap is returned.
- Given a due item, when reminder job runs repeatedly, then duplicate notifications are controlled.

Member tasks

- ☐ Implement ranking as a pure/testable service with named factors.
- ☐ Implement shortage and report calculation helpers.
- ☐ Implement reminder scheduling/job logic with idempotency.

Handoff: Member 5 exposes services; Members 1/2 render reasons/results.

S3-M6-3 | Delivery resilience, data performance and Codecov gate

Branch: s3/m6/release-quality | Reviewer: Member 2 | 4 points | Integrates to: Gitea main

Depends on: All Intermediate integrations and CI

Goal: As the Integration Lead, I want resilient deployment, database/integration tests and meaningful Codecov gates so that Sprint 3 is near-complete rather than only feature-rich.

Acceptance checks

- Given slow/unavailable external service, when production-like tests run, then core Tutor Mx journeys remain available.
- Given realistic database size, when critical queries run, then measured times meet the recorded target after indexing/tuning.
- Given a PR, when coverage changes, then Codecov project/patch checks follow the team's stated policy.

Member tasks

- ☐ Add integration failure/timeout regression and fallback checks.
- ☐ Measure/tune critical database queries and record results for Coordinator.
- ☐ Set Codecov flags/status policy (team target, not official mark threshold).

Handoff: Integration Lead uses this as Sprint 3 release gate; Coordinator writes methodology/testing evidence.

Sprint 3: integration and review gate

- [] Every Member starts from the latest Gitea main and hands over only their own tested branch.
- [] Assigned reviewer reproduces the issue-specific checks and checks page 7/8 evidence.
- [] Integration Lead integrates approved branches into main one at a time and reruns the affected regression/full gate.
- [] If Gitea Actions is unavailable because the lecturer runner is offline, record the blocker and local evidence honestly; do not mark it green.
- [] After integration, run the relevant three-role/public-environment smoke test.
- [] Update the GitHub deployment mirror and manual Cloudflare Production artifact only from approved Gitea main when a release is required.
- [] Issue is Done only when integrated behavior passes; code merely existing on a Member branch is not Done.

SPRINT 3 SPECIFIC RELEASE CHECK

Verify Intermediate ranking/bulk/import/report/notification/search flows and first-use guidance without re-running account role onboarding.

Sprint 4: plan, order and handovers

30 September-11 October 2026 | *Finish claimed Advanced scope, harden and release*

DO NOT REPEAT EARLIER SPRINTS

Sprint 4 starts from the latest approved Gitea main. Do not recreate Auth0 onboarding, core role routing, existing schema/API/UI foundations or manual tutor-ID workflows. Add only the new scope and hardening listed below.

- Implement only demonstrable Advanced scope: scenarios, collaboration/conflict handling, swaps, locks, audit/restore and release resilience.
- Do not refactor working auth/database/API layers merely for novelty.
- Final release evidence must prove the approved Gitea main matches the deployed public system.

Member	Sprint ownership
Member 1	Authentication, role security, notifications/search and shared release checks
Member 2	Organiser interface, allocation board, approvals, reports and scenarios
Member 3	Tutor dashboard, availability, timesheets, excuses, imports and swaps
Member 4	Student overflow plus shared responsive and accessible interface
Member 5	Handwritten Express API, validation, business rules, security and API tests
Member 6	Neon PostgreSQL, Prisma migrations, external integration, data engine, CI/deployment
EVERY MEMBER Before handover: use page 7 for automated tests, page 8 for local browser verification, page 9 for the lecturer-runner rule, and page 12 for the Global Definition of Done.	

Sprint 4 - Member 1

Authentication, role security, notifications/search and shared release checks

REQUIRED BEFORE HANDOVER

See page 7 (run tests), page 8 (check the feature locally), page 9 (Actions runner rule) and page 12 (Global Definition of Done).

CARRY FORWARD

Start from the latest Gitea main and reuse the working Sprints 1-3 implementation. The three cards below are the new work for this sprint.

S4-M1-1 | Production account and session hardening

Branch: s4/m1/auth-hardening | Reviewer: Member 5 | 4 points | Integrates to: Gitea main

Depends on: Production-like environment and full role matrix

Goal: As a Tutor Mx user, I want robust account/session behavior so that final release access is secure and understandable under edge cases.

Acceptance checks

- Given expired/revoked/missing session, when a protected action runs, then re-authentication is required without exposing data.
- Given reset/delete/sign-out boundaries, when completed, then stale tabs/tokens cannot continue protected work.
- Given wrong role/direct API request, when attempted, then 403/401 behavior remains consistent.

Member tasks

- ☐ Audit all account/session flows in production-like build.
- ☐ Fix stale-session, redirect-loop and wrong-role edge cases.
- ☐ Review browser storage and remove sensitive custom persistence.

Handoff: Member 5 fixes backend auth issues; Integration Lead requires green session regression.

S4-M1-2 | Notification/search final fixes and release smoke

Branch: s4/m1/shared-release-smoke | Reviewer: Member 5 | 4 points | Integrates to: Gitea main

Depends on: Final feature scope and Coordinator's severity-one/two list

Goal: As the team, we want search, notifications and the public role entry paths smoke-tested so that final release does not hide critical breakage.

Acceptance checks

- Given clean Organiser/Tutor/Student accounts, when public app/API flows run, then entry, search and notifications work on the exact release candidate.
- Given missing/deleted notification target, when opened, then a safe state is shown.
- Given role-restricted search, when queried, then no forbidden result leaks.

Member tasks

- ☐ Run clean-browser smoke on account, search and notification paths.
- ☐ Fix assigned severity-one/two shared defects.
- ☐ Add regression tests for every fixed defect.

Handoff: Integration Lead re-runs smoke after integration; Coordinator records release evidence.

S4-M1-3 | Final auth/security regression and failure rehearsal

Branch: s4/m1/security-regression | Reviewer: Member 5 | 4 points | Integrates to: Gitea main

Depends on: S4-M1-1, S4-M1-2 and final integrated Gitea main build

Goal: As the Integration Lead, I want a repeatable final auth/security failure rehearsal so that the release tag is not approved on verbal confidence alone.

Acceptance checks

- Given all required account actions and role routes, when final automated/manual regression runs, then every result is recorded against the release candidate.
- Given 401/403/cross-user cases, when tested, then no private response body is returned.
- Given temporary auth/network failure, when rehearsed, then recovery behavior is controlled.

Member tasks

- ☐ Run and record the full auth/role/ownership test matrix.
- ☐ Rehearse one controlled auth/network failure.
- ☐ Open/fix/re-test defects assigned to Member 1.

Handoff: Integration Lead uses this before final Gitea main release decision.

Sprint 4 - Member 2

Organiser interface, allocation board, approvals, reports and scenarios

REQUIRED BEFORE HANDOVER

See page 7 (run tests), page 8 (check the feature locally), page 9 (Actions runner rule) and page 12 (Global Definition of Done).

CARRY FORWARD

Start from the latest Gitea main and reuse the working Sprints 1-3 implementation. The three cards below are the new work for this sprint.

S4-M2-1 | Draft allocation scenarios, compare and publish

Branch: s4/m2/scenario-publish | Reviewer: Member 4 | 4 points | Integrates to: Gitea main

Depends on: Member 5 scenario API and Member 6 engine/schema

Goal: As an organiser, I want draft allocations side by side and an explicit publish action so that I can compare choices before changing live work.

Acceptance checks

- Given two drafts, when compared, then changed assignments, shortages, hours and objective summaries are visible.
- Given a draft, when edited, then live allocations do not change.
- Given locked assignments, when proposals run, then locks remain unchanged.

Member tasks

- ☐ Build draft list/create/rename/delete and comparison UI.
- ☐ Clearly label draft versus live data.
- ☐ Show differences and lock status.

Handoff: Member 5/6 guarantee transactional publish; Member 1 notifications carry changes.

S4-M2-2 | Concurrent organiser presence and conflict resolution

Branch: s4/m2/concurrent-board | Reviewer: Member 4 | 4 points | Integrates to: Gitea main

Depends on: Member 5 concurrency API and Member 6 version/audit data

Goal: As an organiser, I want to see other organisers and resolve conflicting edits so that one person's work is not silently overwritten.

Acceptance checks

- Given two organiser sessions, when both open the same board, then presence or last-update information is visible.
- Given both edit the same allocation/version, when the second save occurs, then a conflict is detected rather than silently overwriting.
- Given conflict, when resolved, then the organiser can refresh/compare/retry and the final chosen state is explicit.

Member tasks

- ☐ Build presence/last-update display.
- ☐ Send expected version with edits and handle conflict response.
- ☐ Build refresh/compare/retry conflict UI.

Handoff: Member 5/6 own version enforcement; Coordinator receives two-browser evidence.

S4-M2-3 | Organiser performance, accessibility and final regression

Branch: s4/m2/organiser-final-quality | Reviewer: Member 4 | 4 points | Integrates to: Gitea main

Depends on: S4-M2-1, S4-M2-2 and final organiser features

Goal: As an organiser, I want the complete board to be fast, keyboard-usable and regression-tested so that the claimed tier is credible in production.

Acceptance checks

- Given realistic large data, when board, ranking, reports and scenario comparison load, then they meet the recorded target or a limitation is logged.
- Given keyboard-only use, when core allocation/bulk/compare/conflict actions run, then they complete successfully.
- Given Basic, Intermediate and claimed Advanced organiser paths, when regression runs, then all pass on release candidate.

Member tasks

- ☐ Measure and improve assigned organiser performance bottleneck.
- ☐ Run keyboard/focus/label/status checks.
- ☐ Run full organiser regression including conflicts/locks/publish.

Handoff: Integration Lead requires green organiser suite; Coordinator records claims/limits.

Sprint 4 - Member 3

Tutor dashboard, availability, timesheets, excuses, imports and swaps

REQUIRED BEFORE HANDOVER

See page 7 (run tests), page 8 (check the feature locally), page 9 (Actions runner rule) and page 12 (Global Definition of Done).

CARRY FORWARD

Start from the latest Gitea main and reuse the working Sprints 1-3 implementation. The three cards below are the new work for this sprint.

S4-M3-1 | Constraint-checked tutor session swaps

Branch: s4/m3/tutor-swaps | Reviewer: Member 6 | 4 points | Integrates to: Gitea main

Depends on: Member 5 swap API and Member 6 allocation engine/data

Goal: As a tutor, I want to propose a session swap that is checked and approved so that changes do not break marks, timetable or weekly-hour rules.

Acceptance checks

- Given an eligible replacement tutor, when a swap is proposed, then all three constraints are checked for affected assignments.
- Given failed constraint, when proposed, then the swap is not applied and the reason is visible.
- Given a valid pending swap, when organiser approves/rejects, then both tutors see the outcome.

Member tasks

- ☐ Build tutor swap proposal/select/reason UI.
- ☐ Show constraint results and pending status.
- ☐ Build or integrate organiser approval view.

Handoff: Member 5/6 enforce rules/transaction; Member 2 integrates organiser approval if needed.

S4-M3-2 | Harden tutor, timetable and timesheet edge cases

Branch: s4/m3/tutor-hardening | Reviewer: Member 6 | 4 points | Integrates to: Gitea main

Depends on: Full Tutor flow and final severity list

Goal: As a tutor, I want boundary cases handled safely so that imports, hours, excuses and timesheets remain trustworthy at release.

Acceptance checks

- Given boundary dates/hours, duplicates and invalid imports, when submitted, then correct validation/history behavior occurs.
- Given returned/disputed/approved timesheet, when actions occur, then only allowed transitions are available.
- Given past/changed assignment excuse, when attempted, then the rule is explained and data stays consistent.

Member tasks

- ☐ Run boundary checklist across dashboard, import, exceptions, work logs, timesheets and excuses.
- ☐ Fix assigned severity-one/two tutor defects.
- ☐ Preserve status/history instead of silent replacement.

Handoff: Member 5/6 receive API/data defects; Coordinator receives final tutor evidence.

S4-M3-3 | Complete tutor regression and payroll/export verification

Branch: s4/m3/tutor-final-tests | Reviewer: Member 6 | 4 points | Integrates to: Gitea main

Depends on: S4-M3-1, S4-M3-2 and release candidate

Goal: As the team, we want the complete tutor workflow and payroll output verified so that final claims match deployed behavior.

Acceptance checks

- Given Basic, Intermediate and claimed Advanced tutor journeys, when final regression runs, then all required paths pass.
- Given known approved work, when payroll export runs, then fields/totals match fixtures and exclude unapproved data.
- Given cross-user/wrong-role requests, when run, then access is denied.

Member tasks

- ☐ Run full tutor automated and clean-browser regression.
- ☐ Verify export schema/totals with known fixtures.
- ☐ Test swap approval plus all invalid constraint cases.

Handoff: Integration Lead uses tutor suite before main; Coordinator writes final evidence from supplied results.

Sprint 4 - Member 4

Student overflow plus shared responsive and accessible interface

REQUIRED BEFORE HANDOVER

See page 7 (run tests), page 8 (check the feature locally), page 9 (Actions runner rule) and page 12 (Global Definition of Done).

CARRY FORWARD

Start from the latest Gitea main and reuse the working Sprints 1-3 implementation. The three cards below are the new work for this sprint.

S4-M4-1 | Overflow race handling and notification polish

Branch: s4/m4/overflow-hardening | Reviewer: Member 2 | 4 points | Integrates to: Gitea main

Depends on: Member 5 atomic claim API and Member 6 constraints

Goal: As students and organisers, we want competing claims and status messages handled correctly so that only one valid outcome is accepted and everyone sees the truth.

Acceptance checks

- Given two students claiming the same item nearly together, when requests finish, then only the permitted winner/pending outcome exists.
- Given closed/filled work, when a stale page submits, then conflict is explained and list refreshes.
- Given approval/rejection/closure, when status changes, then only affected users receive correct notifications.

Member tasks

- ☐ Run two-session claim race tests with Member 5/6 fixtures.
- ☐ Fix stale-page/conflict UI and refresh behavior.
- ☐ Remove duplicate/irrelevant notifications.

Handoff: Member 5/6 fix atomicity/constraint defects; Coordinator receives race-test evidence.

S4-M4-2 | Final shared responsive and accessible visual system

Branch: s4/m4/shared-polish | Reviewer: Member 2 | 4 points | Integrates to: Gitea main

Depends on: All final role screens integrated

Goal: As any user, I want consistent, responsive and accessible screens so that Tutor Mx works without confusing visual or interaction differences.

Acceptance checks

- Given common phone/tablet/desktop widths, when every primary role page renders, then no critical control or information is clipped.
- Given keyboard-only use, when core journeys run, then focus is visible and logical.
- Given forms/statuses, when read by assistive technology, then labels, errors and changes are announced appropriately.

Member tasks

- ☐ Audit and fix shared navigation, forms, tables/cards and state components.
- ☐ Run contrast, label, focus and status-announcement checks.
- ☐ Run common-width screenshots/checks for all role pages.

Handoff: Members 1-3 adopt shared fixes; Coordinator records accessibility/responsive evidence.

S4-M4-3 | Student/shared UI final regression

Branch: s4/m4/student-final-tests | Reviewer: Member 2 | 4 points | Integrates to: Gitea main

Depends on: S4-M4-1, S4-M4-2 and release candidate

Goal: As the team, we want student, notification, search and shared UI regression green so that the final app remains usable after all integrations.

Acceptance checks

- Given Basic/Intermediate student paths, when final regression runs, then list, claim, status and notification cases pass.
- Given slow/error responses, when tested, then loading/fallback/retry states remain understandable.
- Given keyboard/common widths, when shared regression runs, then core paths pass.

Member tasks

- ☐ Run full student/shared automated regression.
- ☐ Run clean-browser slow/error/fallback checks.
- ☐ Fix/re-test defects assigned to Member 4.

Handoff: Integration Lead uses this before main; Coordinator writes final evidence from supplied results.

Sprint 4 - Member 5

Handwritten Express API, validation, business rules, security and API tests

REQUIRED BEFORE HANDOVER

See page 7 (run tests), page 8 (check the feature locally), page 9 (Actions runner rule) and page 12 (Global Definition of Done).

CARRY FORWARD

Start from the latest Gitea main and reuse the working Sprints 1-3 implementation. The three cards below are the new work for this sprint.

S4-M5-1 | Advanced handwritten API workflows

Branch: s4/m5/advanced-api | Reviewer: Member 1 | 4 points | Integrates to: Gitea main

Depends on: Member 6 advanced engine/schema and Basic/Intermediate API

Goal: As the advanced frontend, I want protected scenario, conflict, swap and audit endpoints so that every claimed feature still uses the handwritten API.

Acceptance checks

- Given draft/compare/publish requests, when called, then drafts stay isolated and publish is authorised/transactional.
- Given stale version/conflicting edit, when saved, then 409-style conflict is returned without overwrite.
- Given swap proposal/approval, when called, then constraints and roles are enforced before live change.

Member tasks

- ☐ Implement scenario/diff/publish endpoints.
- ☐ Implement version/conflict/presence support endpoints.
- ☐ Implement swap and audit/restore endpoints.

Handoff: Members 2/3 connect UI; Member 6 supplies engine/history transactions.

S4-M5-2 | API security, design and deployment hardening

Branch: s4/m5/api-hardening | Reviewer: Member 1 | 4 points | Integrates to: Gitea main

Depends on: All final endpoints and production-like environment

Goal: As the school, we want a secure, available and maintainable API so that final deployment does not expose data or depend on Gitea mainnet shortcuts.

Acceptance checks

- Given malformed/oversized/duplicate/unauthorised requests, when sent, then validation, idempotency/limits and permissions protect state.
- Given logs/errors, when inspected, then tokens, database credentials and private payloads are not exposed.
- Given deployment/rollback, when rehearsed, then the API remains externally available or can return to the prior known release.

Member tasks

- ☐ Audit all endpoints for auth, ownership, validation and secrets.
- ☐ Add pagination/limits/idempotency where required.
- ☐ Remove development bypasses and unsafe logs.

Handoff: Integration Lead requires green security/deployment checks; Coordinator records architecture/deployment evidence.

S4-M5-3 | API load, availability and final regression

Branch: s4/m5/api-final-tests | Reviewer: Member 1 | 4 points | Integrates to: Gitea main

Depends on: S4-M5-1, S4-M5-2 and realistic Member 6 data

Goal: As the team, we want measured API performance and complete regression so that final API rubric claims are supported by evidence.

Acceptance checks

- Given realistic load/data, when critical endpoints are measured, then results meet recorded targets or limitations are explicit.
- Given all Basic, Intermediate and claimed Advanced routes, when full regression runs, then success, boundary, permission and failure cases pass.
- Given dependency/database slow failure, when tested, then health/error behavior is controlled.

Member tasks

- ☐ Run load/timing checks on allocation, search, report, scenario and publish routes.
- ☐ Fix assigned high-impact performance defects.
- ☐ Run full API/security/concurrency regression.

Handoff: Integration Lead uses this before main; Coordinator records API performance/testing evidence.

Sprint 4 - Member 6

Neon PostgreSQL, Prisma migrations, external integration, data engine, CI/deployment

REQUIRED BEFORE HANDOVER

See page 7 (run tests), page 8 (check the feature locally), page 9 (Actions runner rule) and page 12 (Global Definition of Done).

CARRY FORWARD

Start from the latest Gitea main and reuse the working Sprints 1-3 implementation. The three cards below are the new work for this sprint.

S4-M6-1 | Explainable school-wide allocation engine with locks

Branch: s4/m6/advanced-allocation-engine | Reviewer: Member 3 | 4 points | Integrates to: Gitea main

Depends on: Final constraints/objectives confirmed by Coordinator/client

Goal: As an organiser, I want a school-wide proposal that respects constraints/locks and explains choices so that I can trust and evaluate the recommendation.

Acceptance checks

- Given courses, tutors and constraints, when engine runs, then it produces a proposal or explicit infeasibility without breaking hard rules.
- Given locked assignments, when engine runs, then locks remain unchanged.
- Given each proposed assignment, when explanation is requested, then chosen factors are stated.

Member tasks

- [] Implement deterministic proposal engine with separate hard constraints and objectives.
- [] Preserve organiser locks and report unmet work.
- [] Generate structured chosen/not-chosen explanations.

Handoff: Member 5 exposes engine; Member 2 renders proposals/explanations.

S4-M6-2 | Audit history, restore and production database recovery

Branch: s4/m6/audit-restore | Reviewer: Member 3 | 4 points | Integrates to: Gitea main

Depends on: All final mutation paths and Member 5 audit endpoints

Goal: As the school, we want every important change recorded and restorable so that the system can explain who changed what and recover from mistakes.

Acceptance checks

- Given allocation/timesheet/claim/scenario changes, when committed, then actor, time, action and before/after reference are recorded.
- Given authorised restore, when an earlier allocation version is selected, then restoration creates a new auditable change rather than deleting history.
- Given database backup/recovery procedure, when rehearsed in a safe environment, then required data returns consistently.

Member tasks

- [] Complete audit event/version schema and transaction hooks.
- [] Implement safe allocation restore service/data operations.
- [] Add indexes/retention choices needed for audit queries.

Handoff: Member 5 exposes authorised endpoints; Coordinator receives recovery/audit evidence.

S4-M6-3 | Final Gitea release gate, deployment mirror and integration resilience

Branch: s4/m6/final-release-gate | Reviewer: Member 3 | 4 points | Integrates to: Gitea main

Depends on: All final code on Gitea main and Integration Lead access

Goal: As the Integration Lead, we want the final release process to prove Gitea main is healthy and the deployment mirror/public site match the approved source.

Acceptance checks

- Full local check/coverage/database/smoke evidence is captured before the final integration/release decision.
- A failed migration, external outage or health check blocks release or uses the agreed controlled fallback/rollback.
- GitHub deployment mirror and manual Cloudflare production upload are refreshed only from approved Gitea main.

Member tasks

- [] Run clean setup, full tests, coverage, database verification and production-like smoke.
- [] Verify Gitea main and release evidence; record lecturer-runner status honestly.
- [] Sync approved Gitea main to the GitHub deployment mirror, rebuild frontend/dist and deploy the final Production artifact to Cloudflare.

Handoff: Integration Lead executes the final Gitea main integration/release check; Coordinator writes submission evidence.

Sprint 4: integration and review gate

- [] Every Member starts from the latest Gitea main and hands over only their own tested branch.
- [] Assigned reviewer reproduces the issue-specific checks and checks page 7/8 evidence.
- [] Integration Lead integrates approved branches into main one at a time and reruns the affected regression/full gate.
- [] If Gitea Actions is unavailable because the lecturer runner is offline, record the blocker and local evidence honestly; do not mark it green.
- [] After integration, run the relevant three-role/public-environment smoke test.
- [] Update the GitHub deployment mirror and manual Cloudflare Production artifact only from approved Gitea main when a release is required.
- [] Issue is Done only when integrated behavior passes; code merely existing on a Member branch is not Done.

SPRINT 4 SPECIFIC RELEASE CHECK

Verify Advanced scope, recovery/failure rehearsals and that the deployed public build corresponds to the final approved Gitea main.

Requirement-to-sprint traceability and official review reminders

Requirement	Where it lives
Separate frontend/backend + handwritten API	S1-M5-1/2 foundation -> S2 workflow endpoints -> S3/4 advanced/hardening.
Auth0 + automatic role onboarding	S1-M1-1/2/3 foundation. Later sprints harden/retest; they do not recreate role/Profile onboarding.
Neon + Prisma	S1-M6-1 foundation -> later migrations extend it only for new workflow/advanced data.
Organiser Tutor Management	S1-M2-2 registered-tutor flow. Later sprints harden/extend; never require organiser Auth0 subject input.
Tutor own-data security	S1-M3 + S1-M5 own-token routes. Later features preserve this ownership model.
Student workflow	S1 protected mock shell -> S2 real overflow/volunteer API -> S3/4 advanced/race/accessibility hardening.
CI / tests / Codecov	Pages 7-9 apply in every sprint; Gitea runner status must be recorded honestly.
Public deployment	Sprint 1 public URL -> later releases sync approved Gitea main to GitHub mirror and deploy fresh artifacts.

Official review/rubric reminder

Review area	Evidence to show
Version control	Useful contributions, traceable branches/commits, Gitea main integration.
Documentation / dev guides	Public/current instructions that match the real stack and workflow.
Work tracker / methodology	Issues, ownership, handovers and sprint evidence actually used.
Tech stack / design	React/Vite + Auth0 + handwritten Express + Prisma/Neon + deployment path clearly motivated.
Stakeholder interaction	Decisions and representative-user findings recorded.
Implementation / quality	Working requirements plus tests, security, accessibility and honest blockers.

Final beginner memory aid

- Gitea main = official team code. GitHub = deployment mirror.
- Member: pull main -> branch -> work -> page 8 local check -> page 7 tests -> commit/push branch -> tell reviewer/Integration Lead.
- Integration Lead: review/integrate approved branch -> rerun tests -> push Gitea main.
- New user: choose role -> Auth0 -> POST /api/auth/onboarding -> matching role/Profile -> correct workspace.
- Existing user: Sign in -> /api/me -> saved workspace. No second role choice.
- Organiser Tutor Management: search signed-up tutor by name/email -> Edit. Never copy an Auth0 User ID.
- Tutor routes: identity comes from the bearer token. Do not design URLs/forms that let a tutor choose another tutor ID.
- Sprint 2-4: build on the latest main; do not reimplement Sprint 1 foundations.
- Tests are local first. An offline lecturer runner is an external blocker, not a reason to skip tests.
- Cloudflare frontend release: build a fresh frontend/dist and upload it as Production after approved integration.

SOURCE NOTE

This handbook was rebuilt from the supplied 95-page Express/Neon sprint handbook and aligned to the integrated Tutor Mx source and recorded Gitea/Auth0/deployment workflow current on 23 August 2026.

Codecov - what it is and why Tutor MX uses it

Beginner explanation of the project coverage workflow

THE SIMPLE IDEA

Codecov does not run our tests. Our tests run first. They produce coverage report files. Codecov receives those files and turns them into a clear dashboard showing which lines of code were covered by tests and which were missed.

Why we use it

1. See how much code is tested

Coverage helps us find areas that need more tests.

2. Separate frontend and backend evidence

Tutor MX uploads the frontend and backend as separate Codecov flags.

3. Track change over time

We keep a dated coverage snapshot for each sprint instead of replacing the previous sprint evidence.

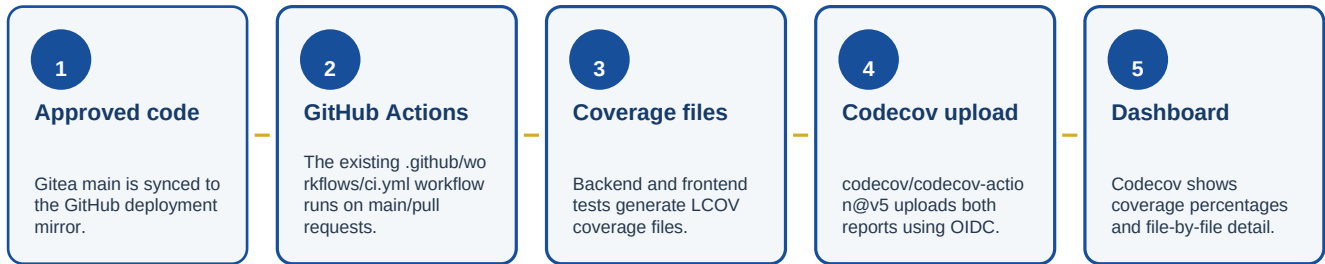
4. Support review evidence

The dashboard and CI upload logs give visible evidence that coverage reports were generated and uploaded.

IMPORTANT: GITEA AND GITHUB ARE DIFFERENT EVIDENCE SOURCES

Gitea **main** remains the official team source of truth. If the lecturer-managed Gitea Actions runner is unavailable, we record that honestly and keep local test evidence. Separately, the approved **GitHub deployment mirror** has its own GitHub Actions workflow, which can run coverage and upload it to Codecov. A successful GitHub mirror run does not mean the unavailable Gitea runner ran.

How Tutor MX sends coverage to Codecov



What the existing CI workflow does

Part	Tutor MX command / file	Codecov setting
Backend	<code>npm run coverage:backend</code> creates <code>./coverage/lcov.info</code>	flag: backend name: tutor-mx-backend
Frontend	<code>npm run test:coverage</code> creates <code>./frontend/coverage/lcov.info</code>	flag: frontend name: tutor-mx-frontend
Uploader	<code>codecov/codecov-action@v5</code>	<code>use_oidc: true</code>
Permission	GitHub workflow permission	<code>id-token: write</code>

WHY OIDC?

The workflow is configured with **`use_oidc: true`** and **`id-token: write`**. This lets the GitHub Actions job authenticate to Codecov with a short-lived identity instead of putting a permanent upload token directly into the workflow file.

How to update Codecov evidence each sprint

Use this same process at the end of Sprint 2, Sprint 3 and Sprint 4

1

1. Sync approved code

Only sync the approved Gitea main state to the GitHub deployment mirror.

2

2. Run the CI workflow

Open GitHub Actions -> CI and let/re-run the workflow for the release candidate.

3

3. Check the three jobs

Backend quality/coverage, frontend quality/coverage, and PostgreSQL migration/API smoke should show their real results.

4

4. Check both Codecov upload steps

Confirm the backend and frontend upload steps completed. Do not rely only on the overall green job if upload errors are allowed not to fail CI.

5

5. Open Codecov

Refresh the tutor-mx-deployment project and record the new frontend and backend coverage.

6

6. Save dated evidence

Take screenshots and label them with the sprint number and date. Keep older sprint screenshots so coverage history is visible.

Sprint 1 baseline recorded on 25 August 2026

Area	Coverage shown in Codecov	Evidence rule
Frontend source	81.58%	Keep as Sprint 1 dated baseline
Backend source	86.37%	Keep as Sprint 1 dated baseline

WHAT TO SAY IN A REVIEW

"Codecov is our coverage reporting dashboard. Our backend and frontend tests generate LCOV reports in GitHub Actions, and the Codecov action uploads them using OIDC. We keep a dated snapshot for every sprint so we can show how coverage changes. Gitea main is still our official source of truth."